Chapter 1

Proof by Tableau

> If controversies were to arise, there would be no more need of disputation between two philosophers than between two accountants. For it would suffice to take their pencils in their hands, and say to each other: Let us calculate.
>
> — Leibniz

An argument is valid when its conclusion cannot be false while its premises are true. For a few propositional letters the definition can be checked by brute force: list every way of making the letters true or false, and look for a row where every premise is true and the conclusion false. There is no such row for modus ponens, from $p \to q$ and p to q:

p	q	$p \to q$	p	q
T	T	T	T	T
T	F	F	T	F
F	T	T	F	T
F	F	T	F	F

The table doubles with every letter, and says nothing about necessity or obligation, where a sentence's truth depends on other worlds than this one. A tableau, or truth tree, does better. Suppose the premises true and the conclusion false, and take the formulas apart by their connectives: $A \wedge B$ puts both A and B on the branch, while $A \vee B$ and $A \to B$ split it in two. A branch holding a formula and its negation is closed (×):

no way of making things true lies along it. If every branch closes, the supposition was impossible and the argument is valid. A branch that stays open describes a counterexample.

1.1 Modus ponens

Here is modus ponens again, as a tableau, with a third premise, that it is cold, which the proof never uses. Denying the conclusion (line 3) and splitting the conditional (line 1) closes both branches at once (figure 1.1).

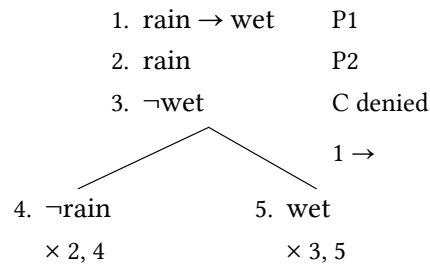


Figure 1.1. Modus ponens, by tableau: both branches close. The premise that it is cold (P3) is never used.

1.2 Necessity

Modal logic reads $\Box A$ as "necessarily A" and $\Diamond A$ as "possibly A". Following Kripke, a model is a set of worlds and a relation R of accessibility between them: $\Box A$ is true at a world when A is true at every world it sees, and $\Diamond A$ when A is true at some world it sees. Each formula in a tableau is now true at a world, w_0 to begin with. A $\Diamond$ opens a new world, and a $\Box$ sends its formula along every arrow already drawn. The axiom of the weakest normal logic, K, is that necessity distributes over the conditional:

$$\Box(p \to q),\ \Box p \vdash \Box q$$

Its tableau (figure 1.2) needs one new world, w_1, and closes on both branches there.

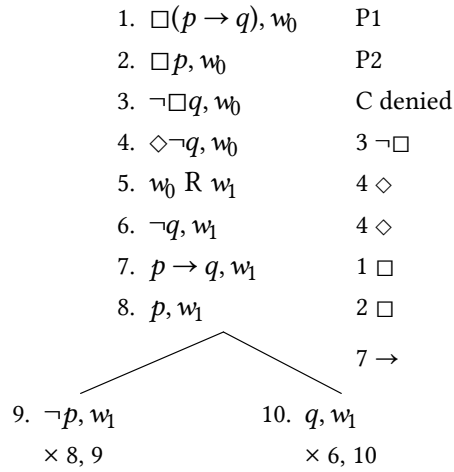


Figure 1.2. Axiom K in the logic K: $\Box(p \to q)$, $\Box p \vdash \Box q$.

1.3 When a branch stays open

What is necessary is surely possible, yet in K $\Diamond p$ does not follow from $\Box p$. Denying $\Diamond p$ gives $\Box \neg p$, no rule of K applies to boxes alone, and the branch stays open (↑). Read off, it is a countermodel: one world that sees none, where every $\Box$ is true and every $\Diamond$ false (figure 1.3).

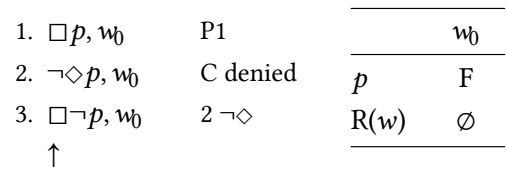


Figure 1.3. From $\Box p$ to $\Diamond p$ in K: an open branch, and its countermodel.

1.4 Obligation

The cure is to require that every world see at least one world: that the relation be serial. This is the logic KD, and its rule D draws a new world where none is there. Read $\Box$ as "it ought to be that" and $\Diamond$ as "it is permitted that", written O and P, and KD becomes the standard de-

1.	O p, w_0	P1
2.	$\neg$P p, w_0	C denied
3.	O $\neg p, w_0$	2 $\neg$P
4.	w_0 R w_1	D
5.	p, w_1	1 O
6.	$\neg p, w_1$	3 O
	$\times$ 5, 6	

Figure 1.4. From O p to P p in KD: the rule D draws w_1, and the tree closes.

ontic logic, the logic of obligation that von Wright set out in 1951. The worlds are now the ideal ones, those in which everything obligatory is done, and seriality says that there is always at least one: no code of conduct demands the impossible. Its axiom is that what is obligatory is permitted, and now the tree closes (figure 1.4).

The same logic proves things that sound wrong. From O p it gives O ($p \vee q$): if I ought to post the letter, then I ought to post it or burn it. This is Ross's paradox, and it is valid in KD, because every ideal world in which the letter is posted is one in which it is posted or burnt. Whether that shows the logic wrong or the English misleading is still argued; the tree only shows what the logic says.

1.5 Trees by machine

The method is mechanical, which is its point. Beth and Hintikka gave it in the 1950s and Smullyan made it standard in *First-Order Logic* (1968): every rule takes a formula apart into smaller ones, so for the propositional logics, and with the proper care for K and KD, a tree always ends, and the method decides validity. For the full logic of quantifiers no method can, as Church and Turing showed in 1936, and a tree may grow without end.

The trees in this chapter were found by a program and drawn by another: `prove` searched for each one, and `tableau` set it out as pic, its formulas in eqn and its countermodels as tables, for troff to set. Each figure is described in words in the PDF, line by line, for a reader who hears it rather than sees it.

The use of such a program is in checking arguments written in prose. Told that the code was audited, and the audit found no bugs, so the code is safe to deploy, `prove` finds the tree of figure 1.5 open. Nothing in the premises speaks of safety at all, and the open branch, read off, is the case the writer passed over: code audited, no bugs found, and still not safe. The program says what would close the gap, a premise that what is audited is safe or that what has no bugs found is, and leaves the writer to say whether either is true.

1.	audited ∧ nobugs	P1
2.	¬safe	C denied
3.	audited	1 ∧
4.	nobugs	1 ∧

↑ audited = T, nobugs = T, safe = F

Figure 1.5. The audited code is safe to deploy? The tree stays open, and its last line is the countermodel: audited and no bugs found, yet not safe.